NumPy

August 29, 2025

NumPy

- Can be used to store data of different forms: 1-d, 2-d, 3-d and so on…
- NumPy is a Python library
- It is used for working with arrays
- It is a Library used for computation and processing of single and multi-dimensional arrays of elements
- Smaller memory consumption and better runtime behaviour
- It is short for "Numerical Python"
- Installation:
  - pip install numpy –> import numpy as np
- Example:
  - values = [20.1, 20.4, 12.3, 43.5, 54.4, 23.5]
  - Convert = np.array(values)
  - print(Convert)
  - print(Convert + 20)

[7]:
```
# To install the library
 # If you're inside Jupyter Notebook / IPython: Prefix with ! or %pip

!pip install numpy
import numpy
```

```
Requirement already satisfied: numpy in c:\users\omkar\anaconda3\lib\site-
packages (2.1.3)
```

Basic Operation

[8]:
```
import numpy as np
```

[47]:
```
# Creating an array

a = np.array([1, 2, 3])
print(a)
```

```
[1 2 3]
```

[13]:
```
type(a)
```

[13]:
```
numpy.ndarray
```

[29]:
```
# Get Dimensions of a numpy array
a.ndim  # It is one dimensional array
```

[29]: 2

[30]:
```
a = np.array([[1, 2, 3]])
print(a)
```

```
[[1 2 3]]
```

[31]:
```
a.ndim
```

[31]: 2

[32]:
```
a = np.array([1, 2, 3], dtype = float)
print(a)
```

```
[1. 2. 3.]
```

[33]:
```
a = np.array([1, 2, 3], dtype = str)
print(a)
```

```
['1' '2' '3']
```

[34]:
```
len(a)
```

[34]: 3

[35]:
```
b = np.array([[1, 2, 3], [4, 5, 6]])
print(b)
```

```
[[1 2 3]
 [4 5 6]]
```

[36]:
```
type(b)
```

[36]: numpy.ndarray

[37]:
```
len(b)
```

[37]: 2

[38]:
```
b.ndim
```

[38]: 2

[39]:
```
c = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(c)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

[40]:
```
len(c)
```

[40]: 3

[41]:
```
c.ndim
```

[41]: 2

[42]:
```
d = np.array([[[[1, 2, 3], [4, 5, 6], [7, 8, 9]]]])
```

[43]:
```
print(d)
```

```
[[[[1 2 3]
   [4 5 6]
   [7 8 9]]]]
```

[44]:
```
len(d)
```

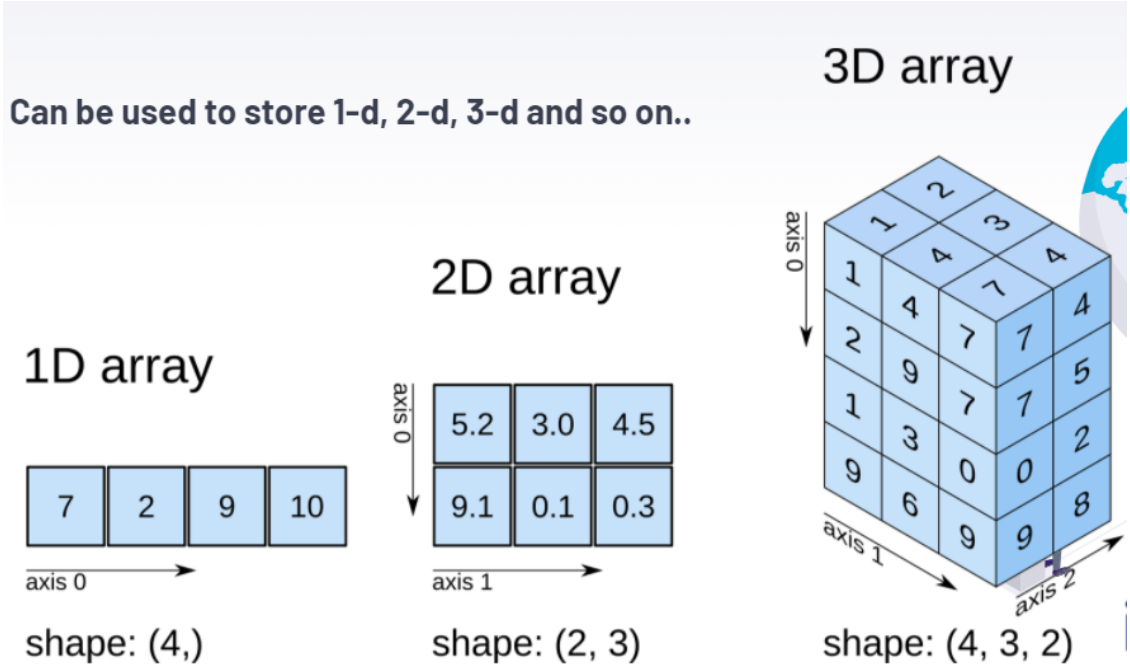
[44]: 1

[45]:
```
d.ndim
```

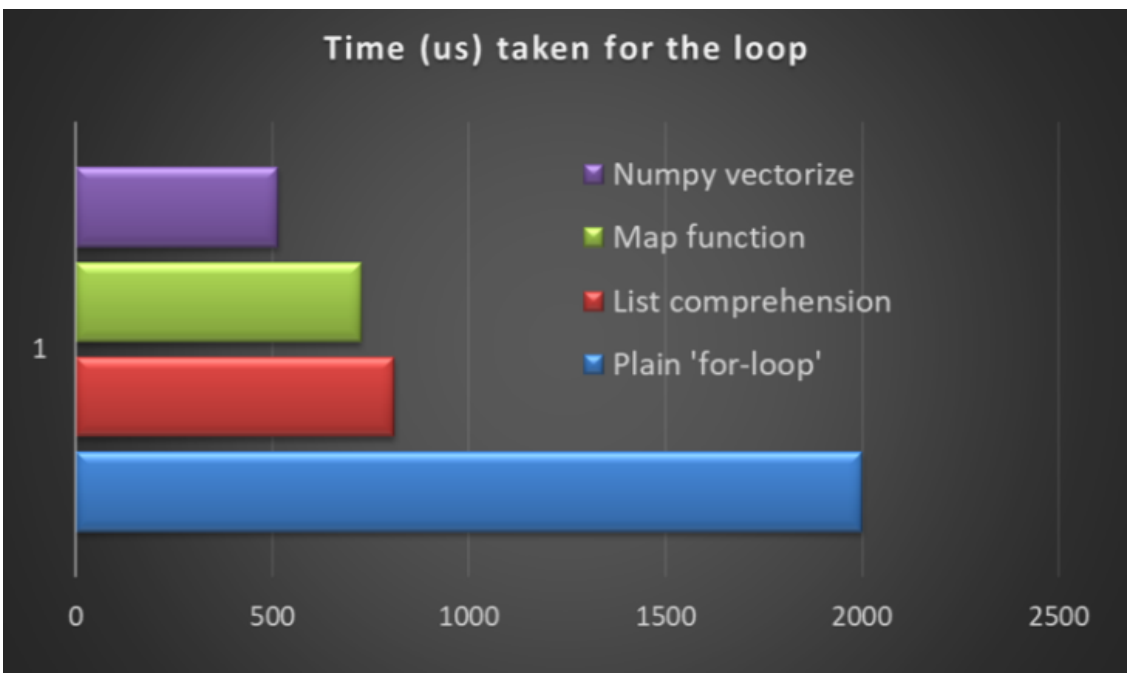
[45]: 4

Why Use NumPy?

- In Traditional Python Programming, we have lists that serve the purpose of arrays, but they are slow to process.
- NumPy aims to provide an array object that is up to 50x faster than traditional Python lists.
- The array object in NumPy is called ndarray, it provides a lot of supporting functions that make working with ndarray very easy.
- Arrays are very frequently used in data science, where speed and resources are very important.



Time taken for the loop:



[78]:
```
a = np.array([1,2,3])
print(a)
```

```
[1 2 3]
```

[79]:
```
# Get a shape of a numpy array

a.shape       # (3,) means it's a 1D array with 3 elements.
```

[79]: (3,)

[80]:
```
a = np.array([[1, 2, 3]])
print(a)
```

```
[[1 2 3]]
```

[81]:
```
a.shape          # (1, 3) means 1 row and 3 columns.
```

[81]: (1, 3)

[84]:
```
b = np.array([[1, 2, 3], [4, 5, 6]])
```

[85]:
```
b.shape        # (2, 3) means 2 rows and 3 columns.
```

[85]: (2, 3)

[86]:
```
b = np.array([1, 2, 3, 4, 5, 6])
```

[87]:
```
b.shape
```

[87]: (6,)

[88]:
```
c
```

[88]:
```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

[89]:
```
c.shape
```

[89]: (3, 3)

[90]:
```
a
```

[90]:
```
array([[1, 2, 3]])
```

[71]:
```
a.dtype  # This tell data type
```

[71]:
```
dtype('int64')
```

[72]:
```
a_int = np.array([1.1, 2.2, 3.3])
```

[73]:
```
a_int
```

[73]: array([1.1, 2.2, 3.3])

[74]:
```
a_int.dtype
```

[74]: dtype('float64')

Accessing/Changing specific elements, rows & columns etc.

[96]:
```
a = np.array([[1,2,3,4,5,6,7], [8,9,10,11,12,13,14]])
print(a)
```

```
[[ 1  2  3  4  5  6  7]
 [ 8  9 10 11 12 13 14]]
```

[97]:
```
a.ndim
```

[97]: 2

[98]:
```
a.shape
```

[98]: (2, 7)

[99]:
```
# Get a specific element [r, c]
print(a[0, 3])
```

```
4
```

[100]:
```
type(a)
```

[100]: numpy.ndarray

[101]:
```
type(a[0, 3])
```

[101]: numpy.int64

[102]:
```
a[1, 4]
```

[102]: np.int64(12)

[103]:
```
print(a[1, 4])
```

```
12
```

[104]:
```
print(a[1, 5])
```

```
13
```

[105]:
```
# Get a specific row from a numpy array
a[0]
```

[105]: array([1, 2, 3, 4, 5, 6, 7])

[106]:
```
a[1]
```

[106]:
```
array([ 8,  9, 10, 11, 12, 13, 14])
```

[108]:
```
a[0, :]
```

[108]:
```
array([1, 2, 3, 4, 5, 6, 7])
```

[109]:
```
a[1, :]
```

[109]:
```
array([ 8,  9, 10, 11, 12, 13, 14])
```

[113]:
```
# Get specific column from numpy array
a[:, 0]
```

[113]:
```
array([1, 8])
```

[114]:
```
a[:, 4]
```

[114]:
```
array([ 5, 12])
```

[124]:
```
a
```

[124]:
```
array([[ 1,  2,  3,  4,  5,  6,  7],
       [ 8,  9, 10, 11, 12, 13, 14]])
```

[125]:
```
a[0, 3]
```

[125]:
```
np.int64(4)
```

[128]:
```
# Change the element
# NumPy array is a mutable in nature
a[0, 3] = 40
print(a)
```

```
[[ 1  2  3 40  5  6  7]
 [ 8  9 10 11 12 13 14]]
```

[130]:
```
# Change the 5th column in 1st row to 50
a [0 ,4] = 50
```

[131]:
```
a
```

[131]:
```
array([[ 1,  2,  3, 40, 50,  6,  7],
       [ 8,  9, 10, 11, 12, 13, 14]])
```

Zeros/Ones method in NumPy

[133]:
```
np.zeros((3, 3))
```

[133]:
```
array([[0., 0., 0.],
       [0., 0., 0.],
       [0., 0., 0.]])
```

[137]:
```
np.zeros((3, 3), dtype = 'int')
```

[137]:
```
array([[0, 0, 0],
       [0, 0, 0],
       [0, 0, 0]])
```

[140]:
```
np.ones((3, 3))
```

[140]:
```
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.]])
```

[141]:
```
np.ones((3, 3), dtype = 'int')
```

[141]:
```
array([[1, 1, 1],
       [1, 1, 1],
       [1, 1, 1]])
```

[142]:
```
np.full((3, 3), 50)
```

[142]:
```
array([[50, 50, 50],
       [50, 50, 50],
       [50, 50, 50]])
```

[143]:
```
var = 30
```

[144]:
```
np.full((3, 3), var)
```

[144]:
```
array([[30, 30, 30],
       [30, 30, 30],
       [30, 30, 30]])
```

[145]:
```
np.random.rand(3, 4)
```

[145]:
```
array([[0.34228799, 0.82051615, 0.77755984, 0.31918736],
       [0.28241856, 0.01985706, 0.44159075, 0.43507685],
       [0.73300767, 0.20573103, 0.63738358, 0.06889238]])
```

[146]:
```
np.random.rand(4, 4)
```

[146]:
```
array([[0.01494338, 0.44954398, 0.55854889, 0.80295773],
       [0.53971204, 0.64454379, 0.35660936, 0.78058389],
       [0.73966146, 0.54217071, 0.12765807, 0.48396201],
       [0.21780252, 0.34022125, 0.32721353, 0.54581485]])
```

[148]:
```
np.random.randint(-4, 1,  size = (3, 3))
```

[148]:
```
array([[ 0, -3, -1],
       [-3, -1, -4],
       [-1, -2,  0]], dtype=int32)
```

[149]:
```
np.random.randint(1, 5,  size = (3, 3))
```

[149]:
```
array([[2, 1, 2],
       [2, 2, 4],
       [4, 4, 4]], dtype=int32)
```

[152]:
```
np.identity(5)            # Identity Matirx: [1 0
                         #                   0 1]
```

[152]:
```
array([[1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       [0., 0., 1., 0., 0.],
       [0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 1.]])
```

[153]:
```
np.identity(3)
```

[153]:
```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

Practice Questions

[155]:
```
# Questin 1
# Extract all the first 3 rows of the last 5 columns in a given numpy 2D array␣
↪'a'?
```

[160]:
```
a= np.array([[1,2,3,4,5,6],
             [10,20,30,40,50,60],
             [12,12,12,12,12,12],
             [11,21,31,41,51,61],
             [13,24,34,44,54,64]])
```

[157]:
```
a
```

[157]:
```
array([[ 1,  2,  3,  4,  5,  6],
       [10, 20, 30, 40, 50, 60],
       [12, 12, 12, 12, 12, 12],
       [11, 21, 31, 41, 51, 61],
       [13, 24, 34, 44, 54, 64]])
```

[158]:
```
# o/p: 2,3,4,5,6
#      20,30,40,50,60
#      12,12,12,12,12
```

[162]:
```
a[ :-2, -5:]
```

[162]:
```
array([[ 2,  3,  4,  5,  6],
       [20, 30, 40, 50, 60],
       [12, 12, 12, 12, 12]])
```

[163]:
```
print(a[0, 1:])   # row 0, columns from index 1 onwards
print(a[1, 1:])   # row 1, columns from index 1 onwards
print(a[2, 1:])   # row 2, columns from index 1 onwards
```

```
[2 3 4 5 6]
[20 30 40 50 60]
[12 12 12 12 12]
```

[164]:
```
a[0:3, 1:]   # rows 0 to 2, columns 1 to end
```

[164]:
```
array([[ 2,  3,  4,  5,  6],
       [20, 30, 40, 50, 60],
       [12, 12, 12, 12, 12]])
```

[165]:
```
# Given a positive number 'n' greater than 2, create a NumPy array of size␣
↪(nxn) with all zeros and ones such that the ones make a shape of "+"

# Examples:
# Input 1:
# 3
# Output 1:
# [[0 1 0]
# [1 1 1]
# [0 1 0]]

# Input 2:
# 5
# Output 1:
# [[0 0 1 0 0]
# [0 0 1 0 0]
# [1 1 1 1 1]
# [0 0 1 0 0]
# [0 0 1 0 0]]
```

[166]:
```
n = 5
```

[167]:
```
# Create an (nxn) array with all zeros
z = np.zeros((n,n), dtype = 'int')
```

[168]:
```
z
```

[168]:
```
array([[0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0]])
```

[169]:
```
# Make the middle row and column all 1s
z[n//2, :] = 1
z[:, n//2]= 1
```

[170]:
```
z
```

[170]:
```
array([[0, 0, 1, 0, 0],
       [0, 0, 1, 0, 0],
       [1, 1, 1, 1, 1],
       [0, 0, 1, 0, 0],
       [0, 0, 1, 0, 0]])
```

Just to help you understand about n//2

[171]:
```
a= np.array([[1,2,3,4,5,6],
             [10,20,30,40,50,60],
             [12,12,12,12,12,12],
             [11,21,31,41,51,61],
             [13,24,34,44,54,64]])
```

[172]:
```
a
```

[172]:
```
array([[ 1,  2,  3,  4,  5,  6],
       [10, 20, 30, 40, 50, 60],
       [12, 12, 12, 12, 12, 12],
       [11, 21, 31, 41, 51, 61],
       [13, 24, 34, 44, 54, 64]])
```

[177]:
```
a[5//2, :] = 1
```

[178]:
```
a[:, 5//2] = 1
```

[179]:
```
a
```

[179]:
```
array([[ 1,  2,  1,  4,  5,  6],
       [10, 20,  1, 40, 50, 60],
       [ 1,  1,  1,  1,  1,  1],
       [11, 21,  1, 41, 51, 61],
       [13, 24,  1, 44, 54, 64]])
```

[180]:
```
a[5//2,:] = 1
print(a)
```

```
[[ 1  2  1  4  5  6]
 [10 20  1 40 50 60]
 [ 1  1  1  1  1  1]
 [11 21  1 41 51 61]
 [13 24  1 44 54 64]]
```

[181]:
```
a[:,n//2] = 1
print(a)
```

```
[[ 1  2  1  4  5  6]
 [10 20  1 40 50 60]
 [ 1  1  1  1  1  1]
 [11 21  1 41 51 61]
 [13 24  1 44 54 64]]
```

Arithmetic Operations with NumPy - Maths + Stats

[183]:
```
a = np.array([1,2,3,4])
```

[184]:
```
a
```

[184]:
```
array([1, 2, 3, 4])
```

[185]:
```
a + 2
```

[185]:
```
array([3, 4, 5, 6])
```

[188]:
```
a-1
```

[188]:
```
array([0, 1, 2, 3])
```

[189]:
```
a*2
```

[189]:
```
array([2, 4, 6, 8])
```

[190]:
```
a/2
```

[190]:
```
array([0.5, 1. , 1.5, 2. ])
```

[191]:
```
a//2
```

[191]:
```
array([0, 1, 1, 2])
```

[186]:
```
a**3
```

[186]:
```
array([ 1,  8, 27, 64])
```

[194]:
```
a
```

[194]: array([1, 2, 3, 4])

[200]:
```
a.min()
```

[200]: np.int64(1)

[201]:
```
print(a.min())
```

```
1
```

[202]:
```
print(np.min(a))
```

```
1
```

[203]:
```
print(a.max())
```

```
4
```

[204]:
```
print(np.max(a))
```

```
4
```

[205]:
```
print(a.mean())
```

```
2.5
```

[206]:
```
print(np.mean(a))
```

```
2.5
```

[207]:
```
a = np.array([[1,2,3,4], [5,6,7,8]])
a
```

[207]:
```
array([[1, 2, 3, 4],
       [5, 6, 7, 8]])
```

[209]:
```
print(np.max(a))
```

```
8
```

[210]:
```
print(a.max())
```

```
8
```

[211]:
```
print(np.min(a))
```

```
1
```

[213]:
```
print(np.mean(a))
```

```
4.5
```

[214]:
```
print(np.sum(a))
```

36

[215]:
```
np.max(a, axis=1)
```

[215]:
```
array([4, 8])
```

[216]:
```
np.max(a, axis=0)
```

[216]:
```
array([5, 6, 7, 8])
```

[217]:
```
np.min(a, axis=0)
```

[217]:
```
array([1, 2, 3, 4])
```

[218]:
```
np.min(a, axis=1)
```

[218]:
```
array([1, 5])
```

[219]:
```
np.sum(a, axis=1)
```

[219]:
```
array([10, 26])
```

[221]:
```
np.sum(a, axis=0)
```

[221]:
```
array([ 6,  8, 10, 12])
```